

Manuel Technique – Projet InLearning

1. Introduction

Le projet **InLearning** est une plateforme de personnalisation de l'apprentissage basée sur une architecture **Data Engineering & IA**.

Elle repose sur un ensemble de composants techniques permettant la collecte, le stockage, le traitement et l'exploitation des données pour fournir :

- Un moteur de recherche et de recommandation (ElasticSearch + SBERT).
 - Une API centralisée (Flask).
 - Des pipelines batch pour l'ingestion et la transformation des données.
 - Une base de données relationnelle (PostgreSQL).
 - Une orchestration via conteneurs Docker.
-

2. Description des composants techniques

2.1 Pipelines Batch

- **Fonction** : Automatiser le traitement des données collectées depuis différentes sources (cours, documents pédagogiques).
 - **Technologies** : Python (Pandas, SQLAlchemy, scripts programmés).
 - **Étapes** :
 1. Extraction → récupération des données brutes.
 2. Transformation → nettoyage, vectorisation (SBERT).
 3. Chargement → stockage des données dans PostgreSQL et indexation dans ElasticSearch.
 - **Planification** : exécutés de manière périodique (cron jobs / Airflow possible en extension).
-

2.2 Conteneurs Docker

- **Fonction** : Garantir la portabilité et la reproductibilité de l'environnement.
 - **Services** définis dans `docker-compose.yml` :
 - **Flask API** → backend applicatif.
 - **PostgreSQL** → base de données relationnelle.
 - **ElasticSearch** → moteur de recherche vectoriel.
 - **Streamlit** → interface utilisateur.
 - **Avantage** : Déploiement rapide et cohérent sur tout type de machine (local, cloud).
-

2.3 API Flask

- **Fonction** : Fournir une couche d'accès centralisée aux données et modèles.
 - **Endpoints principaux** :
 - `/add_course` → ajouter un nouveau cours.
 - `/search` → rechercher un contenu par mot-clé ou vecteur.
 - `/recommend` → générer une recommandation personnalisée.
 - **Sécurité** : authentification JWT (optionnel).
 - **Test** : via `pytest` ou `curl` (`curl -X POST http://localhost:5000/add_course ...`).
-

2.4 Base PostgreSQL

- **Fonction** : Stocker les informations structurées sur les utilisateurs, cours et parcours.
- **Schéma simplifié** :
 - `users (id, nom, email, préférences)`
 - `courses (id, titre, description, source)`
 - `progress (user_id, course_id, status, score)`
- **Connexion** : assurée via SQLAlchemy dans Flask et les pipelines batch.
- **Avantage** : intégrité des données + facilité d'intégration avec BI (Power BI, Streamlit).

2.5 Clé API Anthropic – Génération de cours et prédiction du niveau

- **Fonction :**
 - Générer automatiquement des cours et exercices adaptés au profil utilisateur.
 - Utiliser les données saisies dans les formulaires Streamlit (âge, objectifs, préférences, niveau actuel).
 - Prédire le niveau de l'étudiant grâce à un prompt calibré et stocker la sortie (niveau estimé, recommandations) en base PostgreSQL.
- **Flux technique :**
 1. **Formulaire utilisateur** → collecte des informations (ex. "je veux apprendre Python pour Data Science, je suis débutant").
 2. **Flask API** → endpoint `/generate_course` envoie ces données au modèle Anthropic via la **clé API**.
 3. **Anthropic API** → génère :
 - un plan de cours personnalisé,
 - une estimation du niveau (débutant, intermédiaire, avancé),
 - des quiz ou exercices.
 4. **PostgreSQL** → sauvegarde des résultats (cours, prédictions, feedback).
 5. **Streamlit** → affichage interactif du parcours d'apprentissage.
- **Sécurité :**
 - La clé API est stockée dans le fichier `.env` :

```
ANTHROPIC_API_KEY=xxxxxxxxxxxxxxxxxxxxxx
```
 - Jamais exposée côté frontend.
- **Exemple d'appel API** (dans Flask) :

```
import os
import anthropic
from flask import request, jsonify
```

```
@app.route('/generate_course', methods=['POST'])
def generate_course():
    user_data = request.json
    client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

    prompt = f"""
    Utilisateur: {user_data['age']} ans, objectif: {user_data['objectif']}, niveau: {user_data['niveau']}
    Génère un plan de cours structuré adapté à ce profil avec exercices.
    """

    response = client.messages.create(
        model="claude-3-sonnet-20240229",
        max_tokens=800,
        messages=[{"role": "user", "content": prompt}]
    )
    return jsonify({"generated_course": response.content})
```

3. Procédures d'installation et de configuration

3.1 Prérequis

- **Système** : Linux / Windows 10+ / MacOS.
- **Logiciels** :
 - Docker & Docker Compose
 - Python 3.12
 - Git

3.2 Installation

1. Cloner le projet :

```
git clone https://github.com/tonrepo/inlearning.git
cd inlearning
```

2. Configurer les variables d'environnement (`.env`) :

```
POSTGRES_USER=inlearning
POSTGRES_PASSWORD=secret
POSTGRES_DB=inlearning_db
FLASK_ENV=development
```

3. Lancer les conteneurs :

```
docker-compose up -d
```

3.3 Configuration

- **PostgreSQL** : accessible via `localhost:5432` .
- **ElasticSearch** : disponible sur `http://localhost:9200` .
- **Flask API** : disponible sur `http://localhost:5000` .
- **Streamlit** : interface sur `http://localhost:8501` .

3.4 Configuration de l'API Anthropic

1. Créer un compte sur Anthropic.
2. Générer une **clé API** et la placer dans le fichier `.env` :

```
ANTHROPIC_API_KEY=sk-xxxxxx
```

3. Vérifier que Flask peut accéder à la clé :

```
echo $ANTHROPIC_API_KEY
```

4. Exécution et utilisation (complété)

-

4. Exécution et utilisation

1. Démarrer les services :

```
docker-compose up
```

2. Vérifier l'état :

```
docker ps
```

3. Accéder à l'API :

Exemple pour ajouter un cours :

```
curl -X POST http://localhost:5000/add_course \  
-H "Content-Type: application/json" \  
-d '{"title": "Python avancé", "description": "Cours sur les décorateur  
s"}'
```

4. Accéder à l'interface Streamlit :

Ouvrir <http://localhost:8501> dans un navigateur.

- **Tester la génération de cours :**

```
curl -X POST http://localhost:5000/generate_course \  
-H "Content-Type: application/json" \  
-d '{"age":24,"objectif":"Apprendre Python pour la Data Science","nivea  
u":"débutant"}'
```

- **Résultat attendu :** un JSON contenant :

- `generated_course` → plan de cours structuré,
- `predicted_level` → niveau estimé,

- `recommendations` → prochaines étapes.
-

5. Bonnes pratiques et maintenance

- **Logs** : consulter avec `docker logs <container_id>` .
- **Sauvegarde BDD** :

```
docker exec -t inlearning_postgres pg_dump -U inlearning inlearning_db > backup.sql
```

- **Mises à jour** :
 - Pull du dépôt Git.
 - Redéploiement avec `docker-compose down && docker-compose up -d` .
-

Ce manuel technique constitue la base pour l'installation, la configuration et l'exploitation de **InLearning**.